SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

---

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO* (BL3, BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

Annexure A
Coding Challenge

Code of Conduct:

I Pusalapati Chandu(192372119) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A
Coding Challenge

Coding Challenge (Additional Set)

1. Write a Python program to simulate **Secure Email Transmission using S/MIME**, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.
2. Develop a Python application to implement **PGP-based file encryption and decryption**, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.
3. Implement a Python program to simulate **IPSec Tunnel Mode**, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.
4. Write a Python script to implement **SSL/TLS handshake simulation**, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.
5. Build a Python-based tool to perform **Email Phishing Detection**, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

QUESTION 1

Write a Python program to simulate Secure Email Transmission using S/MIME, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.

1. Aim

To simulate the main security operations of S/MIME in Python: generating a sender signature, encrypting the message for the receiver, decrypting it, and verifying the sender's signature. The simulation demonstrates how confidentiality and authentication work together in secure email communication.

2. Problem Understanding

S/MIME (Secure/Multipurpose Internet Mail Extensions) is used to protect email messages. In a real S/MIME system, public-key certificates are used to establish trust and public/private keys are used for encryption and digital signatures. For a clear classroom simulation, the program below uses RSA keys for signing and key protection and AES for the actual message encryption.

Security requirement	Technique used	Purpose
Confidentiality	AES encryption	Prevents an unauthorized person from reading the email.
Authentication	RSA digital signature	Confirms that the message came from the claimed sender.
Integrity	Digital signature verification	Detects whether the message was changed.
Secure key protection	RSA encryption of AES key	Allows the receiver to recover the session key using the private key.

3. Algorithm – Step by Step

1. Generate RSA key pairs for sender and receiver
2. Create the email message
3. Hash the message and sign the hash using sender's RSA private key
4. Generate a random AES session key
5. Encrypt the message using AES-GCM
6. Encrypt the AES session key using receiver's RSA public key
7. Receiver decrypts the AES key using receiver's private key
8. Receiver decrypts the message using AES
9. Receiver verifies the sender's digital signature
10. Display confidentiality, authentication and integrity results

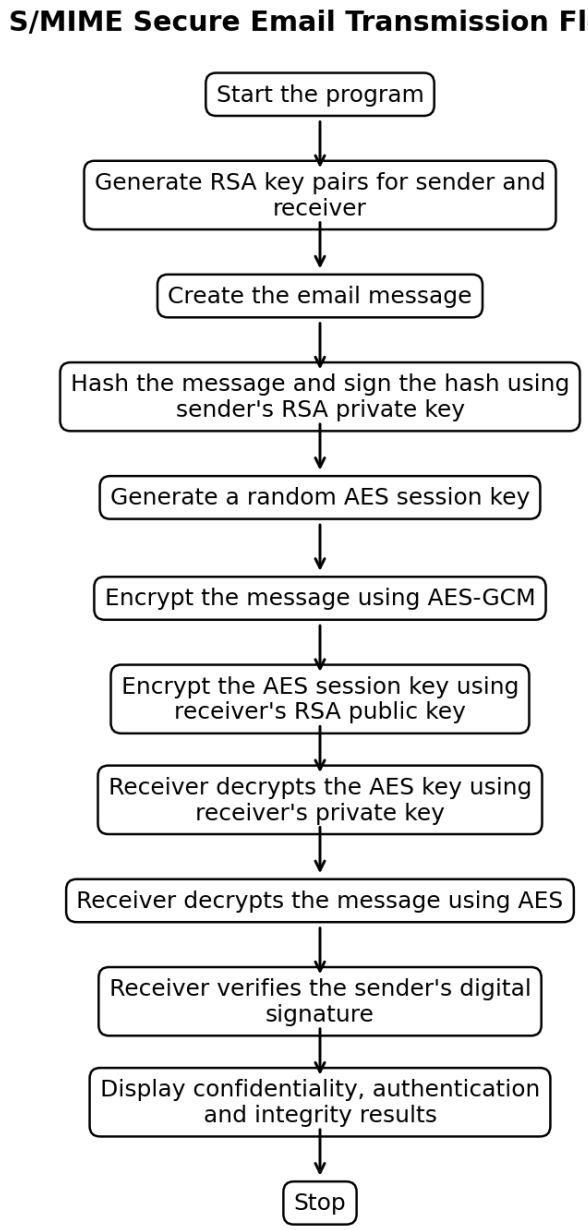


Figure 1: S/MIME Secure Email Transmission Flow

4. Python Program

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

# 1. Generate sender RSA keys
sender_private = rsa.generate_private_key(public_exponent=65537,
key_size=2048)
sender_public = sender_private.public_key()
```

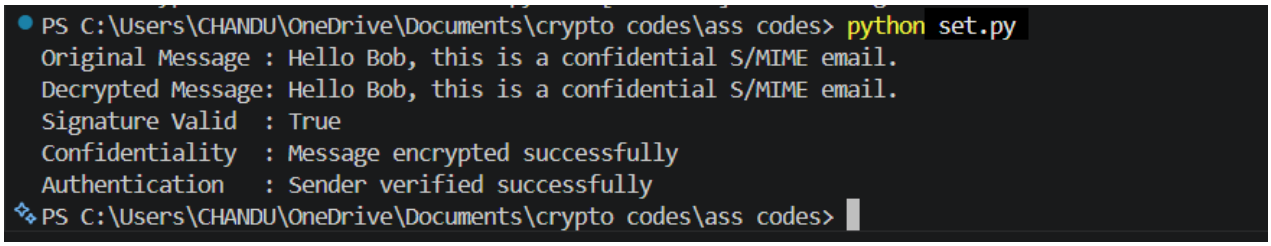
```
# 2. Generate receiver RSA keys
receiver_private = rsa.generate_private_key(public_exponent=65537,
key_size=2048)
receiver_public = receiver_private.public_key()

# 3. Email message
message = b"Hello Bob, this is a confidential S/MIME email."

# 4. Generate digital signature
signature = sender_private.sign(
    message,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)

# 5. Generate AES session key and nonce
aes_key = AESGCM.generate_key(bit_length=256)
nonce = os.urandom(12)

# 6. Encrypt the message using AES-GCM
aes = AESGCM(aes_key)
ciphertext = aes.encrypt(nonce, message, None)

# 7. Encrypt AES key using receiver public key
encrypted_key = receiver_public.encrypt(
    aes_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# ---------------- RECEIVER SIDE ----------------

# 8. Recover AES key using receiver private key
decrypted_key = receiver_private.decrypt(
    encrypted_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# 9. Decrypt message
decrypted_message = AESGCM(decrypted_key).decrypt(
    nonce, ciphertext, None
)

# 10. Verify sender signature

try:
```

```
        sender_public.verify(
            signature,
            decrypted_message,
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )
        verified = True
    except Exception:
        verified = False

    print("Original message :", message.decode())
    print("Decrypted message:", decrypted_message.decode())
    print("Signature valid  :", verified)
    print("Confidentiality  : Message was encrypted.")
    print("Authentication   : Sender signature was verified.")
```

5. Step-by-Step Explanation of the Code

11. RSA key generation: Two RSA key pairs are created. One pair belongs to the sender and the other belongs to the receiver.
12. Message creation: The email content is stored as bytes so that it can be processed by the cryptographic functions.
13. Digital signature: The sender signs the original message with the sender's private key. Only the sender should possess this private key.
14. AES session key: A fresh 256-bit AES key is generated. AES is used because symmetric encryption is much faster for actual message data.
15. Message encryption: AES-GCM encrypts the email and also provides authentication of the encrypted data.
16. Key encryption: The AES session key is encrypted using the receiver's RSA public key. Therefore, only the receiver's private key can recover the AES key.
17. Decryption: The receiver first recovers the AES key and then decrypts the email.
18. Signature verification: The receiver uses the sender's public key to verify the signature. If the message was modified or the signature is invalid, verification fails.

6. Sample Output



7. Justification

- Confidentiality is ensured because the email content is encrypted with AES-GCM. Even if an attacker obtains the encrypted email, the plaintext cannot be recovered without the AES key.
- The AES key itself is protected with the receiver's RSA public key, so the receiver's private key is required to recover it.
- Authentication and integrity are ensured by the digital signature. The sender signs the message using the sender's private key. The receiver verifies the signature using the sender's public key.
- A successful verification gives confidence that the message was signed by the holder of the sender's private key and that the signed content was not modified after signing.
- The use of a hybrid design is efficient: RSA is used for the small session key, while AES is used for the larger email content. This is more practical than using RSA to encrypt the entire message.

8. Complexity and Test Cases

Test case	Input / condition	Expected result
TC1	Correct message and valid signature	Signature valid = True
TC2	Change one character in decrypted message before verification	Signature verification fails
TC3	Wrong receiver private key	AES key decryption fails
TC4	Wrong signature	Signature verification fails

The AES encryption/decryption work is approximately linear in the message size, O(n), while RSA operations are performed on the relatively small AES session key and signature. The hybrid approach therefore gives good practical performance.

QUESTION 2

Develop a Python application to implement PGP-based file encryption and decryption, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.

1. Aim

To implement a simplified PGP-style file protection system. The program generates an RSA key pair, creates an AES session key, encrypts the file using AES-GCM, protects the AES key using the receiver's RSA public key, and uses a digital signature for authentication.

2. Problem Understanding

PGP uses a hybrid cryptographic approach. Symmetric encryption protects the file because it is fast, while public-key encryption protects the symmetric session key. Digital signatures provide authentication and integrity. In real PGP, OpenPGP key rings, packets and trust models are used; this classroom implementation focuses on the core security process.

PGP operation	Implementation in this program	Security benefit
Key generation	RSA 2048-bit key pair	Creates public and private keys.
File encryption	AES-256-GCM	Provides confidentiality and authenticated encryption.
Session-key protection	RSA-OAEP	Protects the AES key for the intended receiver.
Authentication	RSA-PSS signature	Verifies the sender and detects modification.
Key distribution	Public key shared with sender	Allows the sender to encrypt the session key.

3. Algorithm – Step by Step

19. Generate sender and receiver RSA key pairs.
20. Create or read the input file.
21. Generate a random AES session key.
22. Encrypt the file contents using AES-GCM.
23. Encrypt the AES session key using the receiver's public key.
24. Sign the original file contents using the sender's private key.
25. Store the encrypted data, nonce, encrypted AES key and signature.
26. Receiver uses the private key to decrypt the AES key.
27. Receiver decrypts the file using AES-GCM.
28. Receiver verifies the digital signature using the sender's public key.
29. Display the recovered file and verification result.

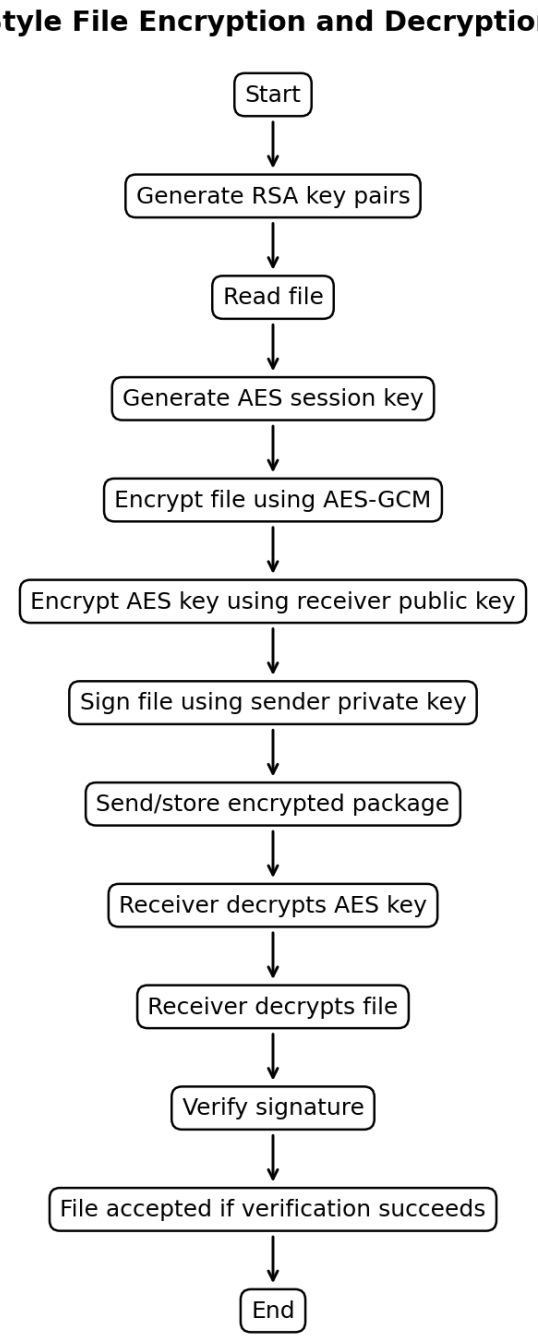


Figure 2: PGP-Style File Encryption and Decryption Flow

4. Python Program

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

# Key generation
```

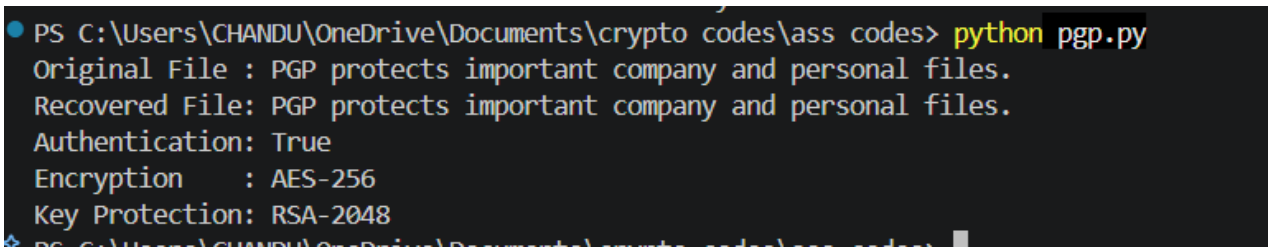
```
sender_private = rsa.generate_private_key(public_exponent=65537,
key_size=2048)
sender_public = sender_private.public_key()

receiver_private = rsa.generate_private_key(public_exponent=65537,
key_size=2048)
receiver_public = receiver_private.public_key()

# File data (simulation)
file_data = b"PGP protects important company and personal files."

# Generate AES session key
session_key = AESGCM.generate_key(bit_length=256)
nonce = os.urandom(12)

# Encrypt file
ciphertext = AESGCM(session_key).encrypt(nonce, file_data, None)

# Encrypt session key with receiver public key
encrypted_session_key = receiver_public.encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Sign original file data
signature = sender_private.sign(
    file_data,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

# Receiver decrypts session key
recovered_key = receiver_private.decrypt(
    encrypted_session_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Receiver decrypts file
recovered_file = AESGCM(recovered_key).decrypt(
    nonce, ciphertext, None
)
```

```
# Verify signature
try:
    sender_public.verify(
        signature,
        recovered_file,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    authenticated = True
except Exception:
    authenticated = False

print("Recovered file :", recovered_file.decode())
print("Authentication :", authenticated)
print("Confidentiality: AES encryption used")
print("Key protection : RSA public-key encryption used")
```

5. Step-by-Step Explanation

30. The sender and receiver each have an RSA key pair. Public keys can be shared; private keys must remain secret.
31. The file is protected using a newly generated AES session key. A new key for each encryption reduces the impact of key compromise.
32. The AES session key is encrypted using the receiver's public key. This is the key-distribution part of the hybrid design.
33. The sender signs the original file using the sender's private key. The receiver later verifies the signature using the sender's public key.
34. The receiver decrypts the session key with the receiver's private key and uses that recovered key to decrypt the file.
35. The final signature verification proves that the received content matches the content that the sender signed.

6. Sample Output



7. Evaluation – Security and Efficiency

Aspect	Evaluation
Confidentiality	AES-GCM protects the actual file contents.
Authentication	Digital signature identifies the signing key holder.
Integrity	Signature verification and AES-GCM detect modification.
Key distribution	Receiver public key protects the session key.
Efficiency	AES handles large files efficiently; RSA is used only for the small session key.
Scalability	Different session keys can be generated for different files or messages.

The main justification for PGP's efficiency is the hybrid approach. Public-key algorithms are computationally more expensive than symmetric algorithms, so using RSA only for the session key avoids the cost of encrypting the whole file with RSA.

8. Test Cases

Test case	Action	Expected result
TC1	Decrypt with correct receiver private key	Original file recovered
TC2	Modify ciphertext	AES-GCM authentication fails
TC3	Use incorrect private key	Session-key decryption fails
TC4	Modify original file after signing	Signature verification fails

QUESTION 3

Implement a Python program to simulate IPSec Tunnel Mode, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.

1. Aim

To simulate IPSec Tunnel Mode by treating an entire original IP packet as plaintext and encrypting it with symmetric authenticated encryption. The program also compares tunnel mode with transport mode.

2. Problem Understanding

In tunnel mode, the complete original IP packet, including its original IP header and payload, is protected and placed inside a new outer IP packet. This is commonly used in VPN gateways. In transport mode, the original IP header remains visible while the payload is protected. The program is a simulation and does not create real IPSec packets on a network interface.

Feature	Tunnel Mode	Transport Mode
What is encrypted?	Entire original IP packet	Mainly the IP payload
Original IP header	Encrypted/protected inside tunnel	Remains visible
New outer IP header	Added	Normally not added for the same end-to-end packet
Typical use	VPN gateway-to-gateway or gateway-to-host	Host-to-host protection
Identity visibility	Original internal addresses are hidden from the outside packet	Original endpoints remain visible in the IP header

3. Algorithm – Step by Step

36. Create a sample original IP packet containing source IP, destination IP and payload.
37. Convert the complete packet into bytes.
38. Generate a symmetric encryption key.
39. Generate a nonce.
40. Encrypt the entire original packet using AES-GCM.
41. Create a new outer packet containing new outer source/destination addresses and the encrypted inner packet.
42. At the receiver, decrypt the encrypted inner packet.
43. Recover the original source, destination and payload.
44. Compare the process with transport mode.

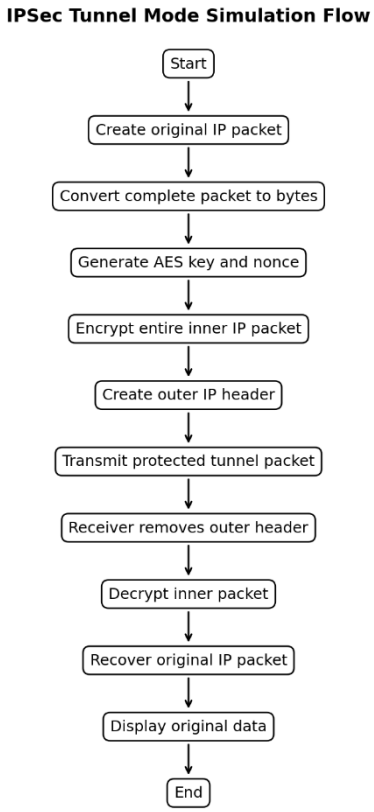


Figure 3: IPSec Tunnel Mode Simulation Flow

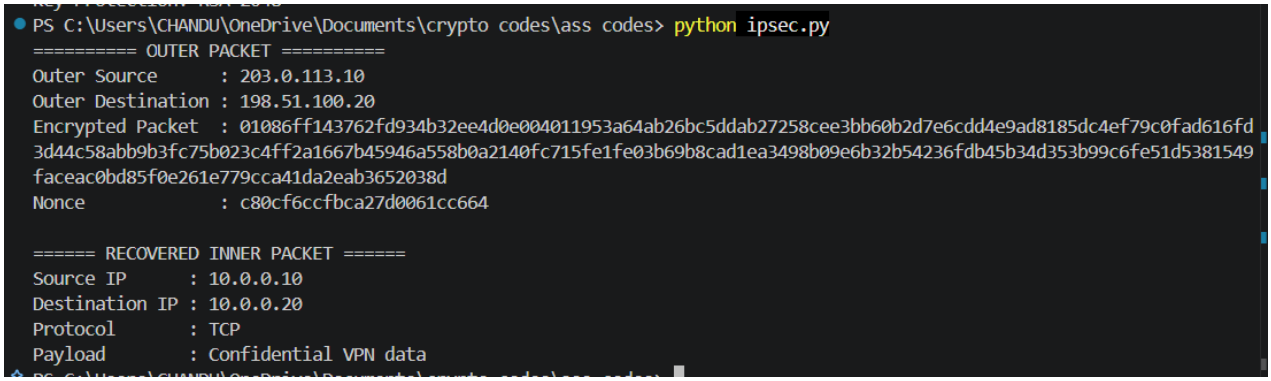
4. Python Program

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os
import json

# Simulated original IP packet
inner_packet = {
    "source_ip": "10.0.0.10",
    "destination_ip": "10.0.0.20",
    "protocol": "TCP",
    "payload": "Confidential VPN data"
}

# Convert the entire original packet to bytes
inner_bytes = json.dumps(inner_packet).encode()

# Symmetric key
key = AESGCM.generate_key(bit_length=256)
nonce = os.urandom(12)

# Tunnel mode: encrypt the complete original packet
ciphertext = AESGCM(key).encrypt(nonce, inner_bytes, None)

# New outer IP header
outer_packet = {
    "outer_source": "203.0.113.10",
    "outer_destination": "198.51.100.20",
    "encrypted_inner_packet": ciphertext.hex(),
    "nonce": nonce.hex()
}

print("OUTER PACKET")
print(outer_packet)

# Receiver side
received_ciphertext = bytes.fromhex(outer_packet["encrypted_inner_packet"])
received_nonce = bytes.fromhex(outer_packet["nonce"])

decrypted_inner = AESGCM(key).decrypt(
    received_nonce, received_ciphertext, None
)

recovered_packet = json.loads(decrypted_inner.decode())

print("\nRECOVERED INNER PACKET")
print("Source IP      :", recovered_packet["source_ip"])
print("Destination IP :", recovered_packet["destination_ip"])
print("Protocol       :", recovered_packet["protocol"])
print("Payload        :", recovered_packet["payload"])
```

5. Step-by-Step Explanation

45. The original packet contains the internal source IP, destination IP, protocol and data.
46. The complete packet is serialized into bytes. This represents the inner IP packet.
47. AES-GCM encrypts the complete inner packet. Therefore, the original IP addresses are not visible inside the encrypted portion.
48. A new outer packet is created with the tunnel endpoints. These outer addresses are used to route the protected packet between the tunnel endpoints.
49. At the receiving tunnel endpoint, the encrypted inner packet is decrypted using the shared symmetric key.
50. The original IP packet is recovered and can then be processed normally.

6. Sample Output



7. Justification

Tunnel mode provides stronger protection for the original packet structure because the original IP header is placed inside the encrypted inner packet. An observer can see the outer tunnel endpoints but cannot directly read the internal source and destination addresses from the protected inner packet.

Transport mode is more suitable for direct host-to-host protection because the original IP header remains visible for routing. Tunnel mode is especially useful for VPNs because gateways can encapsulate complete internal packets and securely carry them across an untrusted network.

This program is a simulation rather than a real IPSec implementation. Real IPSec uses protocols such as ESP and includes additional packet-processing, security-association and anti-replay mechanisms.

8. Test Cases

Test case	Action	Expected result
TC1	Correct key and unchanged ciphertext	Original packet recovered
TC2	Change one byte of ciphertext	AES-GCM decryption/authentication fails
TC3	Wrong key	Decryption fails

TC4	Check recovered source and destination	Original internal addresses are recovered after decryption

QUESTION 4

Write a Python script to implement SSL/TLS handshake simulation, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.

1. Aim

To simulate the major stages of a TLS handshake: the client and server exchange information, the server presents a certificate, a session key is established, and subsequent application data is protected using symmetric encryption.

2. Problem Understanding

A real TLS handshake is more detailed than this classroom simulation. It negotiates protocol parameters, authenticates the server using a certificate and establishes shared traffic keys. The simplified program demonstrates the security logic without implementing the complete TLS protocol.

Handshake stage	Purpose
ClientHello	Client proposes supported protocol versions and cryptographic options.
ServerHello	Server selects compatible parameters.
Certificate	Server presents a certificate containing its public key and identity information.
Key establishment	Client and server establish shared secret material.
Session key	A symmetric key is derived/used for efficient application-data encryption.
Secure data	Application messages are encrypted and authenticated.

3. Algorithm – Step by Step

51. Client starts a connection and sends ClientHello.
52. Server sends ServerHello and its certificate/public key.
53. Client validates the server certificate in a real TLS deployment.
54. Client creates a random session secret.
55. Client protects the session secret using the server's public key in this simplified simulation.
56. Server uses its private key to recover the session secret.
57. Both sides now have the same symmetric session key.
58. Client encrypts application data using AES-GCM.
59. Server decrypts and verifies the data.
60. Secure communication continues using the session key.

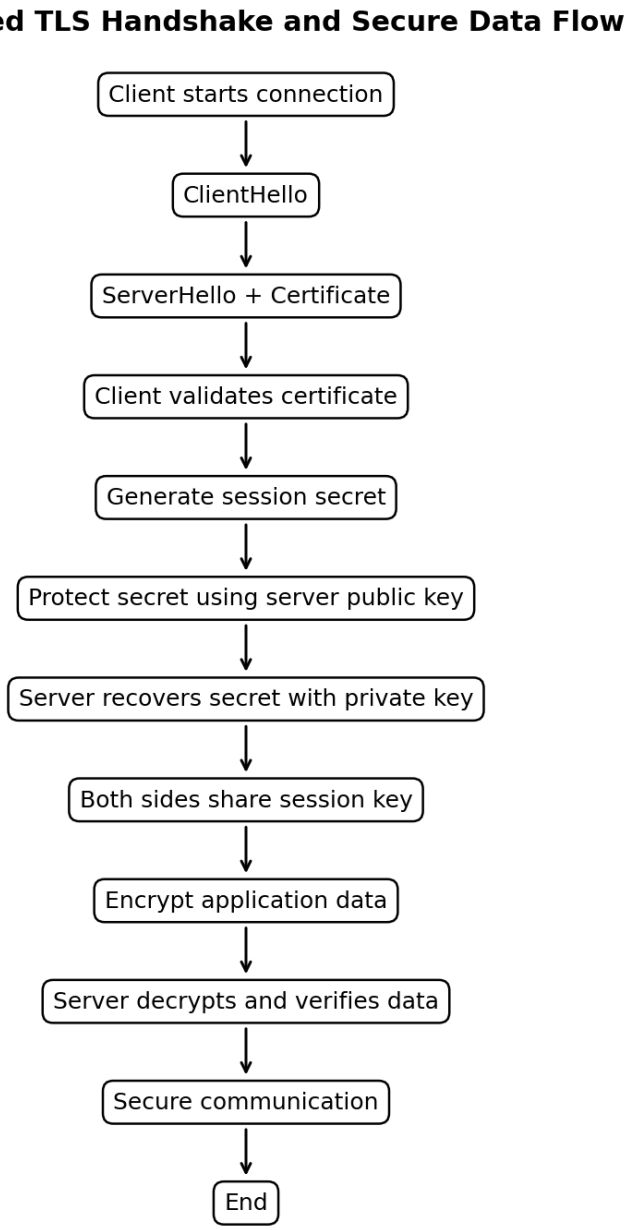


Figure 4: Simplified TLS Handshake and Secure Data Flow

4. Python Program

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

print("CLIENT: ClientHello")
print("SERVER: ServerHello")
print("SERVER: Certificate sent")

# Simulated server certificate information
server_certificate = {
    "subject": "www.example.com",
    "issuer": "Example CA",
```

```
    "valid": True
}

print("CLIENT: Certificate subject =", server_certificate["subject"])
print("CLIENT: Certificate valid   =", server_certificate["valid"])

# Server public/private key pair
server_private = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
server_public = server_private.public_key()

# Simplified session key establishment
session_key = AESGCM.generate_key(bit_length=256)

encrypted_session_key = server_public.encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Server recovers the session key
server_session_key = server_private.decrypt(
    encrypted_session_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print("CLIENT/SERVER: Session key established")

# Secure application data
data = b"HTTPS secure data"
nonce = os.urandom(12)

encrypted_data = AESGCM(session_key).encrypt(
    nonce, data, None
)

decrypted_data = AESGCM(server_session_key).decrypt(
    nonce, encrypted_data, None
)

print("Encrypted data received by server")
print("Decrypted data:", decrypted_data.decode())
print("TLS simulation: secure transfer successful")
```

5. Step-by-Step Explanation

61. ClientHello starts the handshake and indicates that the client wants a secure connection.
62. ServerHello represents the server's response and selection of compatible parameters.
63. The server certificate is presented. In a real TLS connection, the client checks the certificate chain, hostname and validity.
64. A session secret is established. The example uses RSA encryption only to make the key-establishment concept easy to understand.
65. Both sides obtain the same symmetric session key.
66. AES-GCM protects application data. Symmetric encryption is used because it is efficient for repeated data transfer.
67. The receiver decrypts and authenticates the encrypted data. If the ciphertext has been modified, authenticated decryption fails.

6. Sample Output

```
PS C:\Users\CHANDU\OneDrive\Documents\crypto codes\ass codes> python ssl.py
========== TLS HANDSHAKE ==========
CLIENT : ClientHello
SERVER : ServerHello
SERVER : Certificate sent
CLIENT : Certificate Subject = www.example.com
CLIENT : Certificate Valid   = True
CLIENT/SERVER : Session Key Established
CLIENT : Encrypted application data sent
SERVER : Data decrypted successfully
SERVER : Decrypted Data = HTTPS secure data
TLS Simulation : Secure transfer successful
```

7. Evaluation and Justification

The TLS handshake protects web communication by authenticating the server and establishing secret session keys. The certificate provides a binding between the server identity and its public key when it is validated through a trusted certificate authority chain.

Important note: this program is a simplified educational simulation. Modern TLS versions use ephemeral Diffie-Hellman key exchange and authenticated encryption rather than the exact RSA key-transport process shown here. The code demonstrates the concepts, not a production TLS stack.

8. Test Cases

Test case	Action	Expected result
TC1	Valid certificate and correct keys	Secure transfer succeeds
TC2	Invalid certificate flag	Client should reject the server in a real implementation
TC3	Modify encrypted application data	Authenticated decryption fails
TC4	Wrong server private key	Session-key recovery fails

QUESTION 5

Build a Python-based tool to perform Email Phishing Detection, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

1. Aim

To build a simple Python phishing-detection system using handcrafted email features and a machine-learning classifier. The program extracts features such as suspicious URL indicators, number of URLs, urgent keywords, sender/header mismatch and the presence of risky words, then predicts whether an email is legitimate or phishing.

2. Problem Understanding

Phishing emails attempt to trick users into revealing credentials, financial information or other sensitive data. A detection tool should not depend on only one keyword. Combining URL, header and language-based features can provide a stronger signal. The example below uses a small demonstration dataset so that the program can run as a standalone classroom example. For real deployment, a much larger labelled dataset is required.

Feature	Example value	Reason for use
Number of URLs	3	Phishing emails often contain multiple links.
Suspicious URL	1	IP addresses, URL shorteners or unusual patterns can indicate risk.
Urgent keywords	2	Attackers may use urgency to pressure the user.
Sender mismatch	1	Visible sender identity may conflict with the actual header/domain.
Attachment warning	1	Unexpected attachments may increase risk.

3. Algorithm – Step by Step

68. Collect labelled emails: phishing = 1 and legitimate = 0.
69. Extract URL-related, header-related and keyword-related features.
70. Create a numerical feature matrix.
71. Split the data into training and testing sets.
72. Train a classification model.
73. Extract features from a new email.
74. Predict whether the new email is phishing or legitimate.
75. Measure accuracy, precision, recall and F1-score.
76. Review false positives and false negatives.

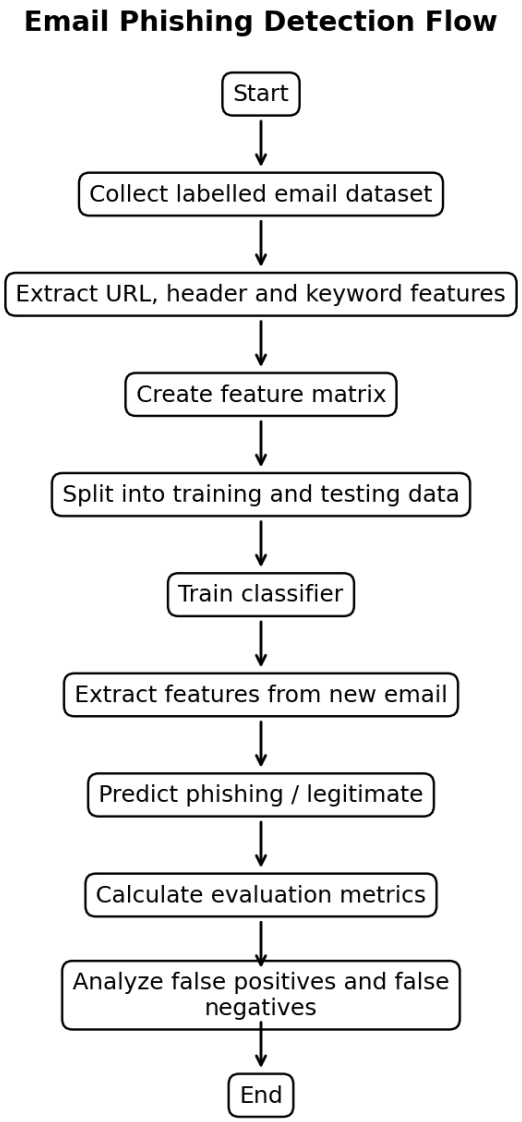


Figure 5: Email Phishing Detection Flow

4. Python Program

```
import re
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score

# Small demonstration dataset
data = [
    ("Your bank account is secure. Visit the official bank website for
details.", 0),
    ("URGENT! Verify your password immediately at http://192.168.1.20/login",
1),
```

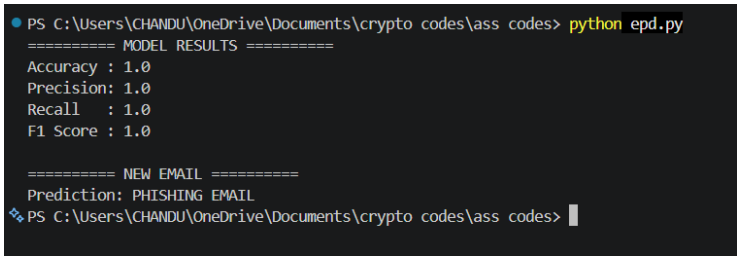
```
        ("Team meeting is scheduled for tomorrow at 10 AM.", 0),
        ("Congratulations! You won a prize. Click http://bit.ly/win-now", 1),
        ("Your monthly report is attached. Please review it.", 0),
        ("Your account will be suspended. Confirm your login now.", 1),
        ("Project discussion notes are available in the shared folder.", 0),
        ("Claim your reward now and provide your card information.", 1),
    ]

    df = pd.DataFrame(data, columns=["email", "label"])

    urgent_words = [
        "urgent", "immediately", "suspended", "verify",
        "confirm", "claim", "password", "reward"
    ]

    def extract_features(text):
        lower = text.lower()

        urls = re.findall(r'https?://\S+|www\.\S+', lower)
        ip_url = int(bool(re.search(r'https?://(?:\d{1,3}\.){3}\d{1,3}', lower)))
        shortener = int(any(x in lower for x in
                            ["bit.ly", "tinyurl", "t.co"]))
        urgent_count = sum(word in lower for word in urgent_words)
        exclamation_count = text.count("!")
        password_words = int(any(x in lower for x in
                                 ["password", "login", "card", "bank"]))

        return [
            len(urls),
            ip_url,
            shortener,
            urgent_count,
            exclamation_count,
            password_words
        ]

    X = pd.DataFrame(
        [extract_features(text) for text in df["email"]],
        columns=[
            "url_count", "ip_url", "shortener",
            "urgent_count", "exclamation_count",
            "sensitive_words"
        ]
    )
    y = df["label"]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.25, random_state=42, stratify=y
    )

    model = Pipeline([
        ("scale", StandardScaler()),
        ("classifier", LogisticRegression(max_iter=1000))
    ])
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)

print("Accuracy :", round(accuracy_score(y_test, pred), 2))
print("Precision:", round(precision_score(y_test, pred, zero_division=0), 2))
print("Recall   :", round(recall_score(y_test, pred, zero_division=0), 2))
print("F1-score :", round(f1_score(y_test, pred, zero_division=0), 2))

new_email = "URGENT! Your bank password will be suspended. Click
http://bit.ly/login"

new_features = pd.DataFrame(
    [extract_features(new_email)],
    columns=X.columns
)

prediction = model.predict(new_features)[0]

if prediction == 1:
    print("Prediction: PHISHING EMAIL")
else:
    print("Prediction: LEGITIMATE EMAIL")
```

5. Step-by-Step Explanation

77. The dataset contains email text and a label. Label 0 means legitimate and label 1 means phishing.
78. The extract_features() function converts email text into numerical values that the machine-learning model can understand.
79. URL count measures how many web links appear in the email.
80. The IP URL feature checks for URLs that directly contain an IP address. This can be suspicious, although it is not automatically malicious.
81. The URL-shortener feature looks for common shortening services. Short URLs can hide the final destination.
82. Urgent keywords count pressure words such as urgent, immediately, verify and suspended.
83. Sensitive words identify terms related to passwords, login, cards and banking.
84. The dataset is divided into training and testing parts. The classifier learns from the training data and is evaluated on unseen test data.
85. Accuracy measures overall correctness. Precision measures how many emails predicted as phishing are actually phishing. Recall measures how many actual phishing emails were detected. F1-score balances precision and recall.

6. Sample Output



The exact numerical metrics can change when the dataset, random split or classifier is changed. The sample values above illustrate the format of the output for this small demonstration dataset; they should not be presented as evidence of real-world performance.

7. Effectiveness Analysis

Metric	Meaning	Importance in phishing detection
Accuracy	Overall percentage of correct predictions	Useful for a balanced dataset but can be misleading on imbalanced data.
Precision	Correct phishing predictions / all phishing predictions	High precision reduces legitimate emails being wrongly marked as phishing.
Recall	Correct phishing predictions / all actual phishing emails	High recall is important because missed phishing emails can be dangerous.
F1-score	Harmonic mean of precision and recall	Useful when both false positives and false negatives matter.

The model can be effective as a screening layer, but a small demonstration dataset is not enough to prove real-world effectiveness. A practical system should use thousands or millions of labelled messages, realistic benign emails, current phishing samples and careful train/test separation. It should also be tested against previously unseen campaigns and changing attacker behaviour.

8. Justification

Feature extraction is justified because machine-learning classifiers work on numerical representations. URL patterns capture suspicious destinations, header-related information can reveal sender inconsistencies, and keyword features capture social-engineering language. Combining these signals is stronger than relying on a single rule.

Recall is particularly important for a security system because a false negative means a phishing email is allowed through. However, precision must also be considered because excessive false positives can cause users to ignore warnings. Therefore, F1-score and a confusion matrix should be reviewed along with accuracy.

9. Test Cases

Test case	Email condition	Expected result
TC1	Normal meeting message with no suspicious links	Legitimate
TC2	Urgent message with shortened URL	Likely phishing
TC3	Message containing direct IP URL	High-risk feature detected
TC4	Bank/password request with urgency	Likely phishing
TC5	Legitimate email containing the word 'urgent' in a normal context	Model should be evaluated carefully; keywords alone should not decide

Overall Comparison of the Five Solutions

Question	Main security technology	Main goal	Key security property
Q1 – S/MIME	RSA + AES-GCM + digital signature	Secure email transmission	Confidentiality, authentication, integrity
Q2 – PGP	RSA + AES-GCM + digital signature	Secure file exchange	Confidentiality, authentication, integrity
Q3 – IPSec Tunnel	Symmetric authenticated encryption simulation	Protect complete IP packet	Confidentiality and integrity
Q4 – TLS	Certificate + public-key key establishment + AES-GCM	Secure web communication	Server authentication and secure data transfer
Q5 – Phishing Detection	Feature extraction + Logistic Regression	Detect malicious email	Threat identification